COMP64401 Module 3: The Description Logic $\mathcal{EL}$

Structured Study Notes for Exam Revision

Generated from uploaded lecture transcripts and slides

Contents

Topic and scope

This lecture block introduces the description logic $\mathcal{EL}$ for knowledge representation and reasoning: its syntax, semantics, reasoning tasks, normal form, consequence-based classification algorithm, lim-

itations/extensions, and its relationship to OWL 2 EL.

It follows the propositional logic part of the course and shows why description logics are better suited for structured conceptual and factual knowledge bases.

1. From propositional logic to description logics

1.1 Recap: what propositional logic covered

The course has already covered:

- **Syntax**: what counts as a formula.
- **Semantics**: valuations assigning truth/falsity to formulas.
- **Reasoning tasks**: what they are, why they matter, and how they relate.
- **A tableau algorithm** for propositional logic, including negation normal form.
- **Use of reasoning in KR applications**.
- **Limitations of propositional logic**.

1.2 Why propositional logic is not enough

The motivating example is weather knowledge. In propositional logic, one might have atoms such as:

$$BS = \text{BlueSky}, \quad Sy = \text{Sunny}, \quad Rg = \text{Raining}$$

and formulas such as:

$$BS \Rightarrow Sy$$

But once we want to talk about **locations** - for example, blue sky in Manchester versus Birmingham - propositional logic forces us into many duplicated atoms:

$$BS_{\text{Man}}, \quad BS_{\text{Bir}}, \ldots$$

and duplicated rules:

$$BS_{\text{Man}} \Rightarrow Sy_{\text{Man}}, \quad BS_{\text{Bir}} \Rightarrow Sy_{\text{Bir}}, \ldots$$

This becomes cumbersome because the same general rule has to be repeated for every object, place, time, or context. Description logics are introduced as a way to represent structured conceptual and factual knowledge more naturally.

1.3 Description logics: three kinds of entities

Description logics distinguish three kinds of entities.

Classes are sets or kinds of things. Examples: `Location`, `Person`, `Weather`, `Rain`, `Temperature`, `Shorts`.

Properties are binary relations between things. Examples: `hasPart`, `isPartOf`, `takes`, `wears`, `isLocatedIn`, `hasWeather`.

Individuals are particular objects/entities. Examples: `Uli`, `Bijan`, `Manchester`, `England`, `UK`.

This lets us separate **conceptual knowledge** from **factual knowledge**.

Conceptual example:

$$\text{Rain} \sqsubseteq \text{Precipitation}$$

Factual examples:

$$\text{Bijan} : \text{Person}$$

$$(\text{Manchester}, \text{England}) : \text{isLocatedIn}$$

Description logics are designed for KR&R applications that need both conceptual and factual knowledge.

2. Syntax of $\mathcal{EL}$ classes

2.1 Formal definition of $\mathcal{EL}$ classes

Let:

- N_C be a set of **class names**.
- N_P be a set of **property names**.
- N_C and N_P are disjoint.

The $\mathcal{EL}$-classes are defined inductively:

1. $\top$ is an $\mathcal{EL}$-class.
2. Every class name $A \in N_C$ is an $\mathcal{EL}$-class.
3. If C, D are $\mathcal{EL}$-classes and $p \in N_P$, then both of the following are $\mathcal{EL}$-classes:

$$C \sqcap D$$

$$\exists p.C$$

2.2 Intuition for the constructors

$\top$

$\top$ is the universal class: everything belongs to it.

The lecturer notes that in OWL, this is called `Thing`.

$C \sqcap D$

This is conjunction/intersection.

Intuition:

$$C \sqcap D$$

means “things that are both C and D.”

Example:

$$\text{BelowF} \sqcap \text{Rain}$$

means things that are both below freezing and rain.

$\exists p.C$

This is an existential restriction.

Intuition:

$$\exists p.C$$

means “things that have some p-successor that is a C.”

Example:

$$\exists \text{wears.Shorts}$$

means things that wear something that is an instance of `Shorts`.

Another nested example:

$$\exists \text{isLocatedIn}.\exists \text{hasWeather}.(\text{BelowF} \sqcap \text{Rain})$$

means things located in some place that has some weather which is both below freezing and rain.

2.3 What $\mathcal{EL}$ does not have at this stage

$\mathcal{EL}$ has:

$$\top, \quad A, \quad C \sqcap D, \quad \exists p.C$$

It does **not** have:

- full disjunction $C \sqcup D$,
- full negation $\neg C$,
- implication as a class constructor.

This is important later because the restricted syntax helps preserve polynomial-time reasoning.

3. Semantics of $\mathcal{EL}$ classes

3.1 Formal definition of an interpretation

An interpretation is:

$$\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$$

where:

- $\Delta^{\mathcal{I}}$ is a non-empty set called the **domain** of $\mathcal{I}$.
- The mapping $\cdot^{\mathcal{I}}$ maps each class name $A \in N_C$ to a set:

$$A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$$

and each property name $p \in N_P$ to a binary relation:

$$p^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$$

The interpretation is extended to complex classes as follows:

$$\top^{\mathcal{J}} := \Delta^{\mathcal{J}}$$

$$(C \sqcap D)^{\mathcal{J}} := C^{\mathcal{J}} \cap D^{\mathcal{J}}$$

$$(\exists p.C)^{\mathcal{J}} := \{x \in \Delta^{\mathcal{J}} \mid \text{there is some } y \in C^{\mathcal{J}} \text{ with } (x, y) \in p^{\mathcal{J}}\}$$

The last definition says that x is in $(\exists p.C)^{\mathcal{J}}$ exactly when x has a p-successor that is an instance of C.

3.2 Terminology

For a class name A:

- $A^{\mathcal{J}}$ is the **extension** of A in $\mathcal{J}$.
- If $b \in A^{\mathcal{J}}$, then b is an **instance** of A in $\mathcal{J}$.

For a property name p:

- If $(b, c) \in p^{\mathcal{J}}$, then c is a p**-successor** of b.

The slide deck explicitly says we should draw interpretations as graphs: elements are nodes, class membership is shown by regions/labels, and property relations are arrows. The diagram on slide/page 12 shows this using arrows for `wears`, `isLocatedIn`, and `hasWeather`.

3.3 Worked interpretation example

The lecture uses the interpretation:

$$\Delta^{\mathcal{J}} = \{a, b, c, d, e, f, g\}$$

with:

$$\top^{\mathcal{J}} = \{a, b, c, d, e, f, g\}$$

$$\text{Weather}^{\mathcal{J}} = \{a, b, c\}$$

$$\text{BelowF}^{\mathcal{J}} = \{a, b\}$$

$$\text{Rain}^{\mathcal{J}} = \{b, c\}$$

$$\text{Shorts}^{\mathcal{J}} = \{g\}$$

$$\text{Person}^{\mathcal{J}} = \{f\}$$

Properties:

$$\text{wears}^{\mathcal{J}} = \{(a, b), (b, b), (f, g)\}$$

$$\text{isLocatedIn}^{\mathcal{J}} = \{(d, e), (e, e), (d, d)\}$$

$$\text{hasWeather}^{\mathcal{J}} = \{(d, a), (d, b), (d, c), (e, a), (e, c)\}$$

Example 1: conjunction

$$(\text{BelowF} \sqcap \text{Rain})^{\mathcal{J}} = \text{BelowF}^{\mathcal{J}} \cap \text{Rain}^{\mathcal{J}}$$

$$= \{a, b\} \cap \{b, c\} = \{b\}$$

So b is the only element that is both below freezing and rain.

Example 2: existential restriction

$$(\exists \text{wears.Shorts})^{\mathcal{J}} = \{f\}$$

because f has a `wears`-successor g, and $g \in \text{Shorts}^{\mathcal{J}}$.

Example 3: weather successor

$$(\exists \text{hasWeather.Weather})^{\mathcal{J}} = \{d, e\}$$

because both d and e have `hasWeather`-successors that are instances of `Weather`.

Example 4: nested existential

$$(\exists \text{isLocatedIn}.\exists \text{hasWeather}.(\text{BelowF} \sqcap \text{Rain}))^{\mathcal{J}} = \{d\}$$

Reason:

1. $(\text{BelowF} \sqcap \text{Rain})^{\mathcal{J}} = \{b\}$.
2. d has a `hasWeather`-successor b.
3. d has an `isLocatedIn`-successor d via the loop (d, d).
4. Therefore d is an instance of the nested class expression.

4. Syntax of $\mathcal{EL}$ axioms, TBoxes, ABoxes, and KBs

4.1 Formal definition of $\mathcal{EL}$ axioms

Let N_I be a set of **individual names**, disjoint from N_C and N_P.

An $\mathcal{EL}$-axiom is one of:

1. **General class inclusion axiom / GCI**:

$$C \sqsubseteq D$$

where C, D are classes.

2. **Class assertion**:

$$b : C$$

where $b \in N_I$ and C is a class.

3. **Property assertion**:

$$(b, c) : p$$

where $b, c \in N_I$ and $p \in N_P$.

4.2 Reading GCIs

A GCI:

$$C \sqsubseteq D$$

can be read as:

- C is a subclass of D,
- C is subsumed by D,
- C implies D,
- all Cs are Ds.

Exam flag / precision warning. The lecturer says not to read $C \sqsubseteq D$ as "C is a subset of D" because C and D are syntactic class expressions, not literal sets. Their **extensions** become sets under a particular interpretation.

4.3 TBox, ABox, knowledge base

A **TBox** is a finite set of GCIs.

An **ABox** is a finite set of class and property assertions.

A **knowledge base** is the union of a TBox and an ABox:

$$\mathcal{K} = \mathcal{T} \cup \mathcal{A}$$

The TBox contains conceptual/terminological knowledge. The ABox contains factual knowledge about individuals.

4.4 Examples

TBox-style axioms:

$$\text{Temperature} \sqsubseteq \text{Weather}$$

$$\text{BelowF} \sqsubseteq \text{Temperature}$$

$$\text{Rain} \sqsubseteq \text{Precipitation}$$

More complex:

$$\text{Person} \sqcap \exists \text{wears.Shorts} \sqcap \exists \text{isLocatedIn.} \exists \text{hasWeather.BelowF} \sqsubseteq \text{ColdLegs}$$

ABox-style axioms:

$$\text{Bijan} : (\text{Person} \sqcap \exists \text{wears.Shorts})$$

$$(\text{Manchester}, \text{England}) : \text{isLocatedIn}$$

$$(\text{England}, \text{UK}) : \text{isLocatedIn}$$

5. Semantics of axioms and models

5.1 Individual names

An interpretation also maps each individual name $b \in N_I$ to an element:

$$b^{\mathcal{J}} \in \Delta^{\mathcal{J}}$$

The lecture notes that different individual names are not automatically required to denote different domain elements in this basic definition.

5.2 Satisfaction of axioms

An interpretation $\mathcal{J}$ satisfies a GCI:

$$C \sqsubseteq D$$

if and only if:

$$C^{\mathcal{J}} \subseteq D^{\mathcal{J}}$$

It satisfies a class assertion:

$$b : C$$

if and only if:

$$b^{\mathcal{J}} \in C^{\mathcal{J}}$$

It satisfies a property assertion:

$$(b, c) : p$$

if and only if:

$$(b^{\mathcal{J}}, c^{\mathcal{J}}) \in p^{\mathcal{J}}$$

An interpretation satisfies an ABox, TBox, or knowledge base if and only if it satisfies **every axiom** in it. Such an interpretation is called a **model** of that ABox/TBox/KB.

Exam flag. The lecturer repeatedly stresses the term **model**: an interpretation that satisfies all axioms. This is distinct from a machine-learning model.

5.3 Worked satisfaction examples

The slide deck gives examples where the interpretation satisfies:

$$\text{BelowF} \sqsubseteq \text{Temperature}$$

because:

$$\{a, b\} \subseteq \{a, b, c\}$$

It also satisfies:

$$\text{Rain} \sqsubseteq \text{Precipitation}$$

because:

$$\{b, c\} \subseteq \{b, c\}$$

And:

$$\text{Person} \sqsubseteq \exists \text{wears.Shorts}$$

because the persons are contained in the extension of things wearing shorts.

The same interpretation does **not** satisfy:

$$\exists \text{locatedIn}.\exists \text{hasWeather}.\text{BelowF} \sqsubseteq \text{Person}$$

because the left-hand extension is not contained in the right-hand extension. The slide gives the counterexample as:

$$\{d, e, f, h\} \not\subseteq \{f, h\}$$

in the extended interpretation.

6. Entailment and reasoning tasks

6.1 Formal definition of entailment

Let $\mathcal{K}$ be an $\mathcal{EL}$ ABox, TBox, or KB, and let α be a GCI, class assertion, or property assertion.

$$\mathcal{K} \models \alpha$$

if and only if every model $\mathcal{I}$ of $\mathcal{K}$ satisfies α.

In words: entailments are axioms that hold in all models of the ABox/TBox/KB.

6.2 Example TBox entailments

From a TBox containing:

$$\text{Temperature} \sqsubseteq \text{WeatherCondition}$$

$$\text{Cold} \sqsubseteq \text{Temperature}$$

we can entail:

$$\text{Cold} \sqsubseteq \text{WeatherCondition}$$

because subclass relationships compose transitively.

Other example entailments include:

$$\text{BelowF} \sqsubseteq \text{WeatherCondition}$$

and statements involving persons, weather, umbrellas, boots, and cold legs, depending on the TBox axioms.

6.3 Modelling warning

The weather examples are deliberately somewhat bad modelling.

The slide explicitly warns that the axioms mix:

- what **should be**,
- and what **is**.

They also ignore exceptions, such as whether everyone feels cold in the same way.

6.4 Syntactic sugar: equivalence

The notation:

$$C \equiv D$$

is syntactic sugar for the two axioms:

$$C \sqsubseteq D$$

and

$$D \sqsubseteq C$$

It does not extend the expressive power of the logic; it is a writing/reading shortcut.

6.5 Main reasoning tasks

Entailment testing

Given a KB $\mathcal{K}$ and an axiom α, test whether:

$$\mathcal{K} \models ? \; \alpha$$

This includes:

$$\mathcal{K} \models ? \; C \sqsubseteq D$$

and:

$$\mathcal{K} \models ? \; b : C$$

Instance retrieval

Given a KB $\mathcal{K}$ and a class C, return all individual names b such that:

$$\mathcal{K} \models b : C$$

Classification

Given a KB $\mathcal{K}$, for each:

$$A, B \in N_C \cup \{\top\}$$

and each individual $b \in N_I$, test:

$$\mathcal{K} \models ? \; A \sqsubseteq B$$

and:

$$\mathcal{K} \models ? \; b : A$$

The result is the **inferred class hierarchy**.

6.6 Inferred class hierarchy

The inferred class hierarchy is:

- a partial order,
- transitive,
- representable as a directed acyclic graph,
- displayable with or without shortcut edges.

Exam flag. The lecturer explicitly says that classification here has very little to do with image classification or machine learning classification. It means computing the inferred class hierarchy.

7. $\mathcal{EL}$ Normal Form / ENF

7.1 Why normal forms matter

Normal forms simplify reasoning algorithms by reducing the number of syntactic cases the algorithm has to handle.

The lecture connects this to negation normal form in propositional logic: normal forms may look less natural to humans, but they make algorithms easier to design and prove correct.

7.2 Formal definition of ENF

An axiom is in $\mathcal{EL}$ **Normal Form** / **ENF** if it is one of:

$$A \sqsubseteq B$$

$$A_1 \sqcap A_2 \sqsubseteq B$$

$$A \sqsubseteq \exists p.B$$

$$\exists p.A \sqsubseteq B$$

$$b : A$$

$$(b, c) : p$$

where:

$$A, A_1, A_2, B \in N_C \cup \{\top\}$$

$$p \in N_P$$

$$b, c \in N_I$$

So in ENF, axioms are very short and mostly atomic.

7.3 Examples in ENF

Examples:

$$\text{Rain} \sqsubseteq \text{Precipitation}$$

$$\text{BelowF} \sqcap \text{Precipitation} \sqsubseteq \text{Slippery}$$

$$\exists \text{teaches}.\top \sqsubseteq \text{Person}$$

$$\text{Professor} \sqsubseteq \exists \text{teaches.UniCourse}$$

$$\text{Uli} : \text{Professor}$$

7.4 Examples not in ENF

Not in ENF:

$$\text{Boots} \sqsubseteq \text{Shoes} \sqcap \text{WarmClothing}$$

because conjunction appears on the right-hand side.

$$\text{Person} \sqcap \exists \text{teaches.UniCourse} \sqsubseteq \text{Professor}$$

because the left-hand conjunction contains a complex existential.

$$\text{Bijan} : (\text{Person} \sqcap \exists \text{wears.Boots})$$

because the class assertion uses a complex class expression.

8. Transforming into ENF

8.1 Basic idea

Any $\mathcal{EL}$ knowledge base can be transformed into an ENF knowledge base by replacing non-conforming axioms with simpler axioms.

For example:

$$A \sqsubseteq B \sqcap C$$

can be replaced by:

$$A \sqsubseteq B$$

$$A \sqsubseteq C$$

This is exact equivalence because:

$$\mathcal{I} \models A \sqsubseteq B \sqcap C$$

if and only if:

$$\mathcal{I} \models A \sqsubseteq B \quad \text{and} \quad \mathcal{I} \models A \sqsubseteq C$$

for the same interpretation $\mathcal{I}$.

8.2 Why exact equivalence is too strong with fresh names

For an axiom such as:

$$A \sqsubseteq \exists p.(B \sqcap C)$$

we might want to introduce a fresh name X and replace it with:

$$A \sqsubseteq \exists p.X$$

$$X \sqsubseteq B \sqcap C$$

But exact equivalence over the same interpretations fails because the original interpretation may not assign any intended meaning to the new class name X.

So the lecture introduces a weaker but appropriate notion: **conservative extension**.

8.3 Conservative extension

Let $\mathcal{K}_1$ and $\mathcal{K}_2$ be two KBs such that all names in $\mathcal{K}_1$ occur in $\mathcal{K}_2$.

$\mathcal{K}_2$ is a **conservative extension** of $\mathcal{K}_1$ if and only if:

1. every model of $\mathcal{K}_2$ is a model of $\mathcal{K}_1$, and
2. every model of $\mathcal{K}_1$ can be extended to a model of $\mathcal{K}_2$.

The extension is about the vocabulary: $\mathcal{K}_2$ may introduce new names, and models of $\mathcal{K}_1$ can be extended by interpreting those new names.

Exam flag. This is the key reason the normal form transformation preserves reasoning over the original vocabulary, even though it introduces fresh auxiliary class names.

9. ENF transformation rules

9.1 Notation

The rules use:

- A for a class name or $\top$,
- C, D for arbitrary $\mathcal{EL}$-classes,
- $\mathbb{C}, \mathbb{D}$ for complex classes that are neither class names nor $\top$,
- X for a fresh class name.

A fresh, different X is introduced for each rule application.

9.2 Rule 1: separate complex left and right sides

$$\mathbb{C} \sqsubseteq \mathbb{D} \rightsquigarrow \mathbb{C} \sqsubseteq X, \quad X \sqsubseteq \mathbb{D}$$

Purpose: introduce a new class name to separate a complex left-hand side from a complex right-hand side.

9.3 Rule 2r: complex right conjunct on the left-hand side

$$C \sqcap \mathbb{D} \sqsubseteq A \rightsquigarrow \mathbb{D} \sqsubseteq X, \quad C \sqcap X \sqsubseteq A$$

Purpose: name the complex right conjunct inside a conjunction on the left-hand side.

9.4 Rule 2l: complex left conjunct on the left-hand side

$$\mathbb{C} \sqcap C \sqsubseteq A \rightsquigarrow \mathbb{C} \sqsubseteq X, \quad X \sqcap C \sqsubseteq A$$

Purpose: name the complex left conjunct inside a conjunction on the left-hand side.

9.5 Rule 3: complex existential on the left-hand side

$$\exists p.\mathbb{D} \sqsubseteq A \rightsquigarrow \mathbb{D} \sqsubseteq X, \quad \exists p.X \sqsubseteq A$$

Purpose: replace the complex filler $\mathbb{D}$ of an existential restriction on the left.

9.6 Rule 4: complex existential on the right-hand side

$$A \sqsubseteq \exists p.\mathbb{D} \rightsquigarrow X \sqsubseteq \mathbb{D}, \quad A \sqsubseteq \exists p.X$$

Purpose: replace the complex filler $\mathbb{D}$ of an existential restriction on the right.

Exam flag. The direction of the definition axiom is different in rules 3 and 4. In rule 3 we use $\mathbb{D} \sqsubseteq X$, but in rule 4 we use $X \sqsubseteq \mathbb{D}$. The lecturer explicitly says this is intentional, not a typo.

9.7 Rule 5: split conjunction on the right-hand side

$$A \sqsubseteq C \sqcap D \rightsquigarrow A \sqsubseteq C, \quad A \sqsubseteq D$$

Purpose: remove conjunctions from the right-hand side.

9.8 Rule 6: complex class assertion

$$b : \mathbb{C} \rightsquigarrow b : X, \quad X \sqsubseteq \mathbb{C}$$

Purpose: replace complex ABox class assertions with atomic ones plus a defining GCI.

10. Worked ENF transformation example

Original TBox:

$$\mathcal{T} = \{\exists p.(A \sqcap \exists p.\top) \sqsubseteq B \sqcap C, \quad B \sqcap \exists r.A \sqsubseteq \exists p.(B \sqcap C)\}$$

10.1 First axiom

Start with:

$$\exists p.(A \sqcap \exists p.\top) \sqsubseteq B \sqcap C$$

Rule 1 separates the complex sides:

$$\exists p.(A \sqcap \exists p.\top) \sqsubseteq X_1$$

$$X_1 \sqsubseteq B \sqcap C$$

Rule 5 splits the right-hand conjunction:

$$X_1 \sqsubseteq B$$

$$X_1 \sqsubseteq C$$

Rule 3 handles the complex existential on the left:

$$\exists p.X_2 \sqsubseteq X_1$$

$$A \sqcap \exists p.\top \sqsubseteq X_2$$

Rule 2r names the complex right conjunct:

$$A \sqcap X_3 \sqsubseteq X_2$$

$$\exists p.\top \sqsubseteq X_3$$

10.2 Second axiom

Start with:

$$B \sqcap \exists r.A \sqsubseteq \exists p.(B \sqcap C)$$

Rule 1 separates complex left and right:

$$B \sqcap \exists r.A \sqsubseteq X_4$$

$$X_4 \sqsubseteq \exists p.(B \sqcap C)$$

Rule 2r names the complex right conjunct:

$$B \sqcap X_5 \sqsubseteq X_4$$

$$\exists r.A \sqsubseteq X_5$$

Rule 4 handles the complex existential on the right:

$$X_4 \sqsubseteq \exists p.X_6$$

$$X_6 \sqsubseteq B \sqcap C$$

Rule 5 splits:

$$X_6 \sqsubseteq B$$

$$X_6 \sqsubseteq C$$

10.3 Resulting ENF TBox

After dropping intermediate non-ENF axioms, the transformed TBox is:

$$\mathcal{T}' = \{X_1 \sqsubseteq B, \quad X_1 \sqsubseteq C, \quad \exists p.X_2 \sqsubseteq X_1, \quad A \sqcap X_3 \sqsubseteq X_2, \quad \exists p.\top \sqsubseteq X_3,$$

$$B \sqcap X_5 \sqsubseteq X_4, \quad \exists r.A \sqsubseteq X_5, \quad X_4 \sqsubseteq \exists p.X_6, \quad X_6 \sqsubseteq B, \quad X_6 \sqsubseteq C\}$$

The transformed $\mathcal{T}'$ is a conservative extension of $\mathcal{T}$.

11. Correctness of ENF transformation

11.1 Theorem

Let $\mathcal{K}$ be an $\mathcal{EL}$ TBox, ABox, or KB. Applying the ENF transformation rules:

1. terminates;
2. results in an $\mathcal{EL}$ TBox/KB $\mathcal{K}'$ that is:
    - of size linear in that of $\mathcal{K}$,
    - a conservative extension of $\mathcal{K}$,
    - in ENF.

11.2 Definition: transformation

For the result of exhaustively applying the ENF transformation rules to $\mathcal{K}$, we say $\mathcal{K}'$ is a **transformation** of $\mathcal{K}$.

"Exhaustively" means no transformation rule is still applicable.

11.3 Corollary: entailment preservation

Let $\mathcal{K}'$ be a transformation of $\mathcal{K}$, and let α be an axiom using only class/property/individual names from $\mathcal{K}$. Then:

$$\mathcal{K} \models \alpha \quad \text{if and only if} \quad \mathcal{K}' \models \alpha$$

So reasoning over $\mathcal{K}'$ gives the same consequences over the original vocabulary as reasoning over $\mathcal{K}$.

Exam flag. The axiom α must use names from the original $\mathcal{K}$, not the auxiliary names introduced in $\mathcal{K}'$.

11.4 Proof sketch

Termination

Each rule application removes a problematic complex form and replaces it with axioms to which fewer transformation rules apply. A rule is applied to each relevant complex subexpression at most once.

Linear size

Each rule application introduces:

- a shorter version of the original axiom, and
- a definition for a subexpression.

This happens at most once per subexpression, and there are only linearly many subexpressions in the original knowledge base. Therefore the transformed KB is linear in size.

Conservative extension

Two directions are needed.

First, every model of $\mathcal{K}'$ is a model of $\mathcal{K}$. For example, for rule 1:

$$\mathbb{C} \sqsubseteq X, \quad X \sqsubseteq \mathbb{D}$$

implies:

$$\mathbb{C}^{\mathcal{J}} \subseteq X^{\mathcal{J}} \quad \text{and} \quad X^{\mathcal{J}} \subseteq \mathbb{D}^{\mathcal{J}}$$

so by transitivity:

$$\mathbb{C}^{\mathcal{J}} \subseteq \mathbb{D}^{\mathcal{J}}$$

hence:

$$\mathcal{J} \models \mathbb{C} \sqsubseteq \mathbb{D}$$

Second, every model of $\mathcal{K}$ can be extended to a model of $\mathcal{K}'$ by interpreting the fresh class names appropriately. For rule 1, we may set:

$$X^{\mathcal{J}} := \mathbb{C}^{\mathcal{J}}$$

or choose another set between $\mathbb{C}^{\mathcal{J}}$ and $\mathbb{D}^{\mathcal{J}}$.

ENF result

Assume $\mathcal{K}'$ were not in ENF. Then it would contain an axiom of a form not allowed in ENF. But then one of the transformation rules would still apply, contradicting exhaustive application.

12. Consequence-based classification algorithm for $\mathcal{EL}$

12.1 Purpose

The algorithm takes a KB $\mathcal{K}$ in ENF and derives all entailed “short” or atomic axioms needed for classification.

It tests, for each:

$$A, B \in N_C \cup \{\top\}$$

and each individual $b \in N_I$, whether:

$$\mathcal{K} \models A \sqsubseteq B$$

and:

$$\mathcal{K} \models b : A$$

It returns the inferred class hierarchy.

12.2 High-level pipeline

The overall process is:

$$\text{Original KB} \longrightarrow \text{ENF Transformer} \longrightarrow \text{Classifier} \longrightarrow \text{Inferred Class Hierarchy}$$

So the normal form transformation is a preprocessing step for the classification algorithm.

13. Classification algorithm

13.1 Input and output

Input:

$$\mathcal{K}$$

an $\mathcal{EL}$ knowledge base in ENF.

Output: the inferred class hierarchy of $\mathcal{K}$.

13.2 Initialisation

Set:

$$\mathcal{K} := \mathcal{K} \cup \{A \sqsubseteq A,\ A \sqsubseteq \top \mid A \in N_C \text{ occurs in } \mathcal{K}\}$$

These are trivial axioms, but they help the rule system derive consequences uniformly.

13.3 Rule application

Apply the following rules until no more rules apply.

Exam flag. A rule is applied only if it changes $\mathcal{K}$, meaning only if it adds something new. The lecturer notes that this condition is implicit in the rule presentation.

14. Consequence rules CR1-CR6

CR1: subsumption transitivity

If:

$$A_1 \sqsubseteq A_2 \in \mathcal{K}$$

and:

$$A_2 \sqsubseteq A_3 \in \mathcal{K}$$

then add:

$$A_1 \sqsubseteq A_3$$

Intuition: subclass chains can be shortcut.

Example:

$$\text{Cold} \sqsubseteq \text{Temperature}$$

$$\text{Temperature} \sqsubseteq \text{WeatherCondition}$$

therefore:

$$\text{Cold} \sqsubseteq \text{WeatherCondition}$$

CR2: conjunction on the left-hand side

If:

$$A \sqsubseteq A_1$$

$$A \sqsubseteq A_2$$

$$A_1 \sqcap A_2 \sqsubseteq B$$

then add:

$$A \sqsubseteq B$$

Intuition: if all As are A_1s and all As are A_2s, and anything both A_1 and A_2 is a B, then all As are Bs.

Example:

$$\text{Blizzard} \sqsubseteq \text{Snow}$$

$$\text{Blizzard} \sqsubseteq \text{BelowF}$$

combined with relevant precipitation facts and:

$$\text{BelowF} \sqcap \text{Precipitation} \sqsubseteq \text{Slippery}$$

allows:

$$\text{Blizzard} \sqsubseteq \text{Slippery}$$

CR3: existential restriction reasoning

If:

$$A \sqsubseteq \exists p.A_1$$

$$A_1 \sqsubseteq B_1$$

$$\exists p.B_1 \sqsubseteq B$$

then add:

$$A \sqsubseteq B$$

Intuition: if As have a p-successor in A_1, and every A_1 is a B_1, and anything with a p-successor in B_1 is a B, then every A is a B.

Example:

$$\text{CleverPerson} \sqsubseteq \exists \text{wears.Umbrella}$$

$$\text{Umbrella} \sqsubseteq \text{RainCover}$$

$$\exists \text{wears.RainCover} \sqsubseteq \text{ComfyPerson}$$

therefore:

$$\text{CleverPerson} \sqsubseteq \text{ComfyPerson}$$

CR4: class assertions inherit along subsumption

If:

$$b : A$$

and:

$$A \sqsubseteq B$$

then add:

$$b : B$$

Example:

$$W : \text{BelowF}$$

$$\text{BelowF} \sqsubseteq \text{Cold}$$

therefore:

$$W : \text{Cold}$$

CR5: conjunction for class assertions

If:

$$b : A$$

$$b : B$$

$$A \sqcap B \sqsubseteq C$$

then add:

$$b : C$$

Intuition: if individual b is both an A and a B, and all things that are both A and B are C, then b is C.

CR6: property assertion plus existential-left GCI

If:

$$(b, c) : p$$

$$c : B$$

$$\exists p.B \sqsubseteq A$$

then add:

$$b : A$$

Intuition: if b has a p-successor c, and c is a B, and everything with a p-successor in B is an A, then b is an A.

15. Worked classification example

The example uses a weather-style KB in ENF with class names such as:

$$\text{Temperature},\ \text{WeatherCondition},\ \text{Precipitation},\ \text{Cold},\ \text{BelowF},\ \text{Rain},\ \text{Snow},\ \text{Blizzard}$$

and auxiliary names:

$$X_1, X_2, X_3, X_4$$

introduced during normalisation.

15.1 Initialisation

The algorithm adds axioms such as:

$$\text{Temperature} \sqsubseteq \text{Temperature}$$

$$\text{Temperature} \sqsubseteq \top$$

$$\text{WeatherCondition} \sqsubseteq \text{WeatherCondition}$$

$$\text{WeatherCondition} \sqsubseteq \top$$

and similarly for auxiliary names like X_1.

15.2 CR1 examples

Using CR1, add:

$$\text{Cold} \sqsubseteq \text{WeatherCondition}$$

$$\text{BelowF} \sqsubseteq \text{Temperature}$$

$$\text{BelowF} \sqsubseteq \text{WeatherCondition}$$

$$\text{Rain} \sqsubseteq \text{WeatherCondition}$$

$$\text{Blizzard} \sqsubseteq \text{Precipitation}$$

15.3 CR2 example

From:

$$\text{Blizzard} \sqsubseteq \text{BelowF}$$

$$\text{Blizzard} \sqsubseteq \text{Precipitation}$$

$$\text{BelowF} \sqcap \text{Precipitation} \sqsubseteq \text{Slippery}$$

add:

$$\text{Blizzard} \sqsubseteq \text{Slippery}$$

15.4 CR3 example

From:

$$\text{CleverPerson} \sqsubseteq \exists \text{wears.Umbrella}$$

$$\text{Umbrella} \sqsubseteq \text{RainCover}$$

$$\exists \text{wears.RainCover} \sqsubseteq \text{ComfyPerson}$$

add:

$$\text{CleverPerson} \sqsubseteq \text{ComfyPerson}$$

15.5 ABox rule examples

Given ABox facts like:

$$(\text{Bijan}, S1) : \text{wears}$$

$$S1 : \text{Shorts}$$

$$\text{Bijan} : \text{Person}$$

$$(\text{Manchester}, W) : \text{hasWeather}$$

$$W : \text{BelowF}$$

$$(\text{Bijan}, \text{Manchester}) : \text{isLocatedIn}$$

The algorithm derives:

$$W : \text{Cold}$$

by CR4.
Then:

$$\text{Bijan} : X_2$$

by CR6, using the fact that Bijan wears $S1$, $S1$ is Shorts, and $\exists \text{wears.Shorts} \sqsubseteq X_2$.
Then:

$$\text{Manchester} : X_4$$

by CR6, after deriving that W is Cold.
Then:

$$\text{Bijan} : X_3$$

by CR6, using Bijan's location relation to Manchester.

Then:

$$\text{Bijan} : X_1$$

by CR5.

Finally:

$$\text{Bijan} : \text{ColdLegs}$$

by CR5, using Bijan's personhood plus the auxiliary class memberships.

Exam flag. The lecturer warns that rule order is not one-and-done. Applying later rules can make earlier rules applicable again.

16. Correctness of the classification algorithm

16.1 Theorem

Let $\mathcal{K}$ be an $\mathcal{EL}$ KB in ENF. Applying the classification algorithm:

1. terminates;
2. results in an $\mathcal{EL}$ KB $\mathcal{K}'$ of size polynomial in that of $\mathcal{K}$;
3. for each $\alpha \in \mathcal{K}'$, we have:

$$\mathcal{K} \models \alpha$$

4. for all $A, B \in N_C \cup \{\top\}$, if:

$$\mathcal{K} \models A \sqsubseteq B$$

then:

$$A \sqsubseteq B \in \mathcal{K}'$$

5. for all $b \in N_I$ and $B \in N_C$, if:

$$\mathcal{K} \models b : B$$

then:

$$b : B \in \mathcal{K}'$$

Point 3 means **no wrong entailments** are added. Points 4 and 5 mean all entailed atomic subsumptions and atomic class assertions are made explicit.

16.2 Soundness and completeness

The lecturer summarises the theorem as:

- the algorithm is **sound**: everything added is entailed;
- the algorithm is **complete for atomic subsumptions and atomic class assertions**: all such entailments are eventually made explicit.

16.3 Complexity corollary

Computing the inferred class hierarchy of an $\mathcal{EL}$ KB can be done in polynomial time.

The lecture explicitly contrasts this with propositional satisfiability, which is NP-complete. The lecturer also notes that human/cognitive complexity and computational complexity differ: the $\mathcal{EL}$ algorithm may look more complicated than the propositional tableau algorithm, but computationally it is better behaved.

17. Proof sketch for classification correctness

17.1 Termination and polynomial size

Let:

- m be the number of class names, including $\top$,
- n be the number of property names,
- i be the number of individual names.

The possible ENF axiom forms are:

$$A \sqsubseteq B$$

$$A_1 \sqcap A_2 \sqsubseteq B$$

$$A \sqsubseteq \exists p.B$$

$$\exists p.A \sqsubseteq B$$

$$b : A$$

$$(b, c) : p$$

The number of possible axioms of these forms is polynomially bounded:

$$m^2, \quad m^3, \quad m^2 n, \quad m^2 n, \quad im, \quad i^2 n$$

so the total is bounded by:

$$m^3 + m^2(1 + 2n) + mi + ni^2$$

Therefore the rules can add at most polynomially many axioms.

Since rules never remove axioms and are applied only when they add something new, the algorithm terminates after polynomially many additions.

17.2 Soundness proof idea

For each rule CRi, prove:

if CRi adds α, then:

$$\mathcal{K} \models \alpha$$

Example: CR1

Take any model $\mathcal{J}$ of $\mathcal{K}$.

If:

$$A_1 \sqsubseteq A_2$$

and:

$$A_2 \sqsubseteq A_3$$

are in $\mathcal{K}$, then:

$$A_1^{\mathcal{J}} \subseteq A_2^{\mathcal{J}}$$

and:

$$A_2^{\mathcal{J}} \subseteq A_3^{\mathcal{J}}$$

By transitivity of subset:

$$A_1^{\mathcal{J}} \subseteq A_3^{\mathcal{J}}$$

so:

$$\mathcal{J} \models A_1 \sqsubseteq A_3$$

Since this holds for every model $\mathcal{J}$, we get:

$$\mathcal{K} \models A_1 \sqsubseteq A_3$$

Example: CR6

Take any model $\mathcal{J}$ of $\mathcal{K}$.

If:

$$(b, c) : p$$

$$c : B$$

$$\exists p.B \sqsubseteq A$$

then:

$$(b^{\mathcal{J}}, c^{\mathcal{J}}) \in p^{\mathcal{J}}$$

$$c^{\mathcal{J}} \in B^{\mathcal{J}}$$

so:

$$b^{\mathcal{J}} \in (\exists p.B)^{\mathcal{J}}$$

Because:

$$(\exists p.B)^{\mathcal{J}} \subseteq A^{\mathcal{J}}$$

we get:

$$b^{\mathcal{J}} \in A^{\mathcal{J}}$$

so:

$$\mathcal{J} \models b : A$$

and therefore:

$$\mathcal{K} \models b : A$$

17.3 Completeness proof idea

The proof for points 4 and 5 is more complicated. The lecture sketches it using a **canonical interpretation** $\mathcal{J}'$ for the saturated KB $\mathcal{K}'$.

Define:

$$\Delta^{\mathcal{J}'} := \{x_A \mid A \in N_C \text{ in } \mathcal{K}\} \cup \{x_b \mid b \in N_I \text{ in } \mathcal{K}\} \cup \{x_\top\}$$

For each class name B:

$$B^{\mathcal{J}'} := \{x_A \mid A \sqsubseteq B \in \mathcal{K}'\} \cup \{x_b \mid b : B \in \mathcal{K}'\}$$

For each property p:

$$p^{\mathcal{J}'} := \{(x_A, x_B) \mid A \sqsubseteq \exists p.B \in \mathcal{K}'\} \cup \{(x_a, x_b) \mid (a, b) : p \in \mathcal{K}'\}$$

For each individual b:

$$b^{\mathcal{J}'} := x_b$$

Then show:

1. $\mathcal{J}'$ is a model of $\mathcal{K}'$.
2. Since $\mathcal{K} \subseteq \mathcal{K}'$, $\mathcal{J}'$ is also a model of $\mathcal{K}$.
3. If $A \sqsubseteq B \notin \mathcal{K}'$, then by construction:

$$A^{\mathcal{J}'} \not\subseteq B^{\mathcal{J}'}$$

so:

$$\mathcal{K} \not\models A \sqsubseteq B$$

4. Similarly for missing class assertions $b : B$.

18. What $\mathcal{EL}$ can express

18.1 Class hierarchies

$\mathcal{EL}$ can express class hierarchies such as:

$$\text{ElectricEngine} \sqsubseteq \text{Engine}$$

and use reasoning to arrange classes automatically in an inferred hierarchy.

18.2 Definitions using necessary and sufficient conditions

The lecture uses the example:

$$\text{Duck} \equiv \text{Bird} \sqcap \exists \text{livesOn.Water} \sqcap \exists \text{soundsLike.Quack}$$

This gives necessary and sufficient conditions for being a duck:

- If something is a duck, it is a bird, lives on water, and sounds like quack.
- If something is a bird, lives on water, and sounds like quack, then it is a duck.

18.3 Domain of properties

$\mathcal{EL}$ can express domains of properties.

Example:

$$\exists \text{isPoweredBy}.\top \sqsubseteq \text{Device}$$

Intuition: anything that is powered by something is a device.

18.4 Multi-dimensional modelling

The lecture presents multi-dimensional modelling as a major benefit of description logics.

Instead of manually building one huge monolithic vehicle hierarchy, we can define vehicle classes in terms of several smaller hierarchies:

- engine/power source,
- terrain,
- location,
- material.

For example, the engine hierarchy can “drive” the vehicle hierarchy. A vehicle can be defined by what powers it, what terrain it moves through, where it is built, and what material it is built from. The slide diagrams on pages 66-69 show the vehicle hierarchy being informed by separate hierarchies such as source/engine and terrain.

The advantage is that changing one hierarchy can automatically affect the inferred vehicle hierarchy through classification.

The lecturer compares this to disentangling concerns in object-oriented programming, but says the benefit is stronger because otherwise the hierarchy would become enormous.

19. Limitations of $\mathcal{EL}$

$\mathcal{EL}$ is intentionally restricted. The lecture lists several things that cannot be expressed in plain $\mathcal{EL}$.

19.1 Disjointness of classes

Cannot say:

$$\text{Person} \sqcap \text{Vehicle} \sqsubseteq \bot$$

So $\mathcal{EL}$ cannot enforce that persons and vehicles are disjoint.

19.2 Range of properties

Can say a domain axiom such as:

$$\exists \text{isPoweredBy}.\top \sqsubseteq \text{Device}$$

but cannot say the range of `isPoweredBy` is `PowerSource`.

19.3 Property hierarchies

Cannot express:

$$\text{hasDaughter} \sqsubseteq \text{hasChild}$$

$$\text{hasChild} \sqsubseteq \text{isRelatedTo}$$

19.4 Transitivity of properties

Cannot express:

$$\text{trans}\big(\text{isLocatedIn}\big)$$

For example, if Salford is located in Greater Manchester and Greater Manchester is located in North West England, plain $\mathcal{EL}$ cannot derive that Salford is located in North West England.

19.5 Individuals as classes

Cannot use an individual as a class.

Example that cannot be expressed in plain $\mathcal{EL}$:

$$\text{Manchester} : \text{City}$$

$$\text{Mancunian} \equiv \exists \text{livesIn}.\{\text{Manchester}\}$$

19.6 Inverse properties

Cannot link:

$$\text{isPoweredBy}$$

and:

$$\text{powers}$$

as inverses.

20. $\mathcal{EL}^{++}$

$\mathcal{EL}^{++}$ extends $\mathcal{EL}$ with several expressive features while preserving polynomial-time reasoning.

It supports:

- disjointness of classes,
- property ranges,
- property hierarchies,
- transitive properties,
- individuals as classes / nominals,
- other extensions used in OWL 2 EL.

But it still does **not** have full disjunction or full negation.

Why not full disjunction?

The lecturer explains that proper disjunction forces reasoning by cases. Reasoning by cases tends to branch, and branching often leads to exponential behaviour. $\mathcal{EL}^{++}$ was designed to keep reasoning polynomial, so full disjunction is excluded.

21. Relationship to propositional logic and other DLs

Core comparison:

- **Propositional logic** has conjunction, disjunction, and negation. Its semantics is based on valuations at a single point. Its main reasoning tasks include satisfiability and validity. Propositional satisfiability is NP-complete.
- $\mathcal{EL}/\mathcal{EL}^{++}$ has conjunction, but no full disjunction and no full negation. In $\mathcal{EL}^{++}$ there is restricted negation-like expressivity through disjointness. Its semantics uses interpretations with many elements. Main reasoning tasks are entailment and classification, and the main reasoning task discussed here is polynomial.

- **Other DLs such as ALC** can include full disjunction and negation. Their complexity can be PSpace, ExpTime, or worse, depending on the logic.

The lecturer stresses that description logics are not all "simpler than propositional logic." Some DLs are less complex; others are more complex.

22. $\mathcal{EL}$, OWL, and applications

22.1 What is OWL?

OWL is a web ontology language.

In computer science, "ontology" is often used as another term for a knowledge base, including both:

- TBox,
- ABox.

However, terminology varies:

- ABox is often called a **knowledge graph**.
- TBox is often called an **ontology**.

The lecturer warns that these terms are not completely settled, so one should check what someone means by "ontology" or "knowledge graph."

22.2 OWL 2 and $\mathcal{EL}^{++}$

OWL was developed in two stages:

- OWL,
- OWL 2.

OWL 2 builds on research in KR&R, especially semantics, entailment, complexity, and algorithms for description logic fragments.

$$\mathcal{EL}^{++}$$

is the logical basis of:

OWL 2 EL

22.3 $\mathcal{EL}$ versus OWL 2

$\mathcal{EL}$:

- is a logic,
- has syntax for axioms, ABoxes, TBoxes, KBs,
- has semantics via interpretations,
- supports reasoning tasks such as entailment and classification.

OWL 2:

- is an ontology language built on $\mathcal{EL}^{++}$,
- has various web-friendly syntaxes,
- has axioms and ontologies,
- inherits semantics from the underlying DL,
- inherits reasoning tasks from the underlying DL,
- adds practical features:
  - annotations,
  - XML datatypes,

- datatype properties,
- imports,
- versioning,
- more expressive variants such as OWL 2 DL.

Exam flag / super important slide note. OWL allows us to distinguish between a class/concept and the labels or terms used for it. Example:

- Class: `Person`
- Label: `Human`
- Biology label: `Homo sapiens`
- German label: `Mensch`

This separates lexical naming from logical meaning.

23. OWL 2 usage

23.1 Knowledge-heavy industries

OWL 2 ontologies are used in knowledge-heavy domains such as:

- biohealth,
- biochemistry.

The lecture mentions BioPortal, which provides access to more than 1,000 ontologies, including mature and widely used ontologies such as:

- SNOMED CT,
- nanoparticle ontologies,
- National Cancer Institute Thesaurus / NCIT.

23.2 Organising taxonomies via definitions

Example ontology fragment:

$$\text{Patient} \equiv \text{Person} \sqcap \exists \text{suffersFrom.Disease}$$

$$\text{Inflammation} \sqsubseteq \text{Disease}$$

$$\text{HeartDisease} \equiv \text{Disease} \sqcap \exists \text{hasLoc.Heart}$$

$$\text{Endocarditis} \equiv \text{Inflammation} \sqcap \exists \text{hasLoc.Endocardium}$$

$$\text{Endocardium} \sqsubseteq \text{Bodypart} \sqcap \exists \text{isPartOf.Heart}$$

$$\text{hasLoc} \circ \text{isPartOf} \sqsubseteq \text{hasLoc}$$

From these, the reasoner derives:

$$\text{Endocarditis} \sqsubseteq \text{HeartDisease}$$

The important point is that the ontology author defines `Endocarditis`, classifies the ontology, and the reasoner places `Endocarditis` in the correct position in the taxonomy. Manual placement is described as time-consuming, error-prone, and hard to maintain.

23.3 Typing individuals

Using the same medical-style TBox, suppose:

$$\text{Bob} : \big(\text{Person} \sqcap \exists \text{suffersFrom}.\big(\text{Inflammation} \sqcap \exists \text{hasLoc}.\text{Endocardium}\big)\big)$$

The reasoner can infer:

$$\text{Bob} : \text{Patient}$$

$$\text{Bob} : \exists \text{suffersFrom}.\text{HeartDisease}$$

$$\text{Bob} : \text{HeDiPatient}$$

where:

$$\text{HeDiPatient} \equiv \text{Person} \sqcap \exists \text{suffersFrom}.\text{HeartDisease}$$

This enables querying for heart disease patients and retrieving Bob even if Bob was not explicitly asserted to be a heart disease patient.

23.4 Linking individuals

OWL 2 property hierarchies can support reasoning over relationships between individuals.

The lecture mentions examples from a cell line ontology, including property-chain-style axioms such as:

$$\text{capableOf} \circ \text{partOf} \sqsubseteq \text{capableOfPartOf}$$

and:

$$\text{endsWith} \circ \text{negativelyRegulates} \sqsubseteq \text{negativelyRegulates}$$

[UNCLEAR: The transcript is garbled around some property names here; the slide gives the general form and examples, but the exact intended property labels may need checking against the recording/slides.]

23.5 Tools

The lecture mentions:

- OWL reasoners,
- ontology editors such as Protégé,
- OWL API,
- OWLready2,
- specialist tools for:
  - explanation of entailments,
  - module extraction,
  - diffing.

Exam/coursework flag. The lecturer says Protégé will be used in coursework.

24. Exam flags and high-value points

24.1 Definitely know

- **Model** = an interpretation satisfying all axioms in the ABox/TBox/KB. The lecturer says they will ask this repeatedly.
- $\mathcal{EL}$-class syntax:

$$\top,\ A,\ C \sqcap D,\ \exists p.C$$

- Interpretation semantics:

$$\top^{\mathcal{J}} = \Delta^{\mathcal{J}}$$

$$(C \sqcap D)^{\mathcal{J}} = C^{\mathcal{J}} \cap D^{\mathcal{J}}$$

$$(\exists p.C)^{\mathcal{J}} = \{x \mid \exists y \in C^{\mathcal{J}}.(x, y) \in p^{\mathcal{J}}\}$$

- Axiom types:

$$C \sqsubseteq D, \quad b : C, \quad (b, c) : p$$

- TBox = finite set of GCIs.
- ABox = finite set of class/property assertions.
- KB = TBox plus ABox.
- Entailment:

$$\mathcal{K} \models \alpha$$

if and only if all models of $\mathcal{K}$ satisfy α.

- Classification means computing the inferred class hierarchy, not ML/image classification.
- ENF axiom forms.
- Conservative extension, especially why fresh names prevent ordinary equivalence.
- ENF transformation theorem:
  - terminates,
  - linear size,
  - conservative extension,
  - in ENF.
- Classification algorithm rules CR1-CR6.
- Classification theorem:
  - terminates,
  - polynomial size,
  - sound,
  - complete for atomic subsumptions and atomic class assertions.
- $\mathcal{EL}$ reasoning is polynomial; propositional satisfiability is NP-complete.
- $\mathcal{EL}^{++}$ extends expressivity while preserving polynomial reasoning.
- OWL 2 EL is based on $\mathcal{EL}^{++}$.
- OWL annotations distinguish classes/concepts from labels/terms.

24.2 Common mistakes / precision traps

- Do not call $C \sqsubseteq D$ "C is a subset of D" without mentioning interpretations/extensions.
- Do not assume fresh auxiliary names have fixed meanings in the original interpretation.
- Do not forget that ENF transformation preserves entailments only over the **original vocabulary**.

- Do not swap the direction of the fresh-name axiom in transformation rules 3 and 4.
- Do not apply consequence rules only once in fixed order; later rule applications can make earlier rules applicable again.
- Do not implement the classification rules naively by scanning every combination in a large ontology; the lecturer warns that practical implementations need clever indexing/optimisation.

25. Unclear / garbled transcript sections to revisit

- [UNCLEAR] The transcript repeatedly says "L", "yellow", "EAL", or similar. These should be $\mathcal{EL}$, confirmed by the slides.
- [UNCLEAR] "Entitlements" in the transcript should be **entailments**.
- [UNCLEAR] "E boxes", "AbeBooks", or similar should be **ABoxes**.
- [UNCLEAR] "DCI/GHCI" should be **GCI**, general class inclusion.
- [UNCLEAR] The transcript says there are "six" and sometimes "seven" transformation rules. The slides number rules 1-6, but rule 2 has left/right variants $2l$ and $2r$, so there are six numbered rules and seven displayed schemas.
- [UNCLEAR] The normal-form transcript mentions a typo in a slide around replacing $A \sqsubseteq \exists p.(B \sqcap C)$. Use the corrected conservative-extension version:

$$A \sqsubseteq \exists p.X, \quad X \sqsubseteq B \sqcap C$$

- [UNCLEAR] The complexity formula in the classification transcript is garbled verbally; the slide formula gives the polynomial bound:

$$m^3 + m^2(1 + 2n) + mi + ni^2$$

up to the usual polynomial-bound interpretation.

- [UNCLEAR] In the OWL limitations transcript, some terms are badly transcribed: "Alta," "out," "oil," and similar refer to **OWL/OWL 2**; "L plus plus" is $\mathcal{EL}^{++}$; "RLC/I'll see" appears to refer to **ALC**.
- [UNCLEAR] The vehicle example in the slide parse says `MotorVehicle Bird poweredBy.Engine`, but the lecturer says this should be a **device** powered by an engine, not a bird. The transcript explicitly corrects this verbally.
- [UNCLEAR] The cell-line ontology property-chain examples are partly garbled in the transcript; revisit the recording if exact property names matter for revision.